Experimental Comparison of Sorting Algorithms

[Your Name]
West University of Timișoara

March 2026

1 Introduction

This report presents an experimental analysis of six fundamental sorting algorithms: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, and Heap Sort. The objective is to verify theoretical time complexities through empirical data across various input sizes (n) and data distributions.

2 Methodology

The algorithms were implemented in Python 3.x. The test suite evaluated performance using:

- **Sizes:** $n \in \{50, 500, 5000, 50000\}$
- **Distributions:** Random, Sorted, Reverse, Nearly Sorted, and Flat (high duplicate count).
- **Environment:** Tests were conducted on a Windows-based system to measure execution time in seconds.

3 Results and Analysis

3.1 General Performance Trends ($n \leq 5,000$)

As shown in Figure 1, there is a massive performance gap between elementary ($O(n^2)$) and efficient ($O(n \log n)$) algorithms. As n increases, the elementary algorithms show a sharp quadratic increase in execution time.

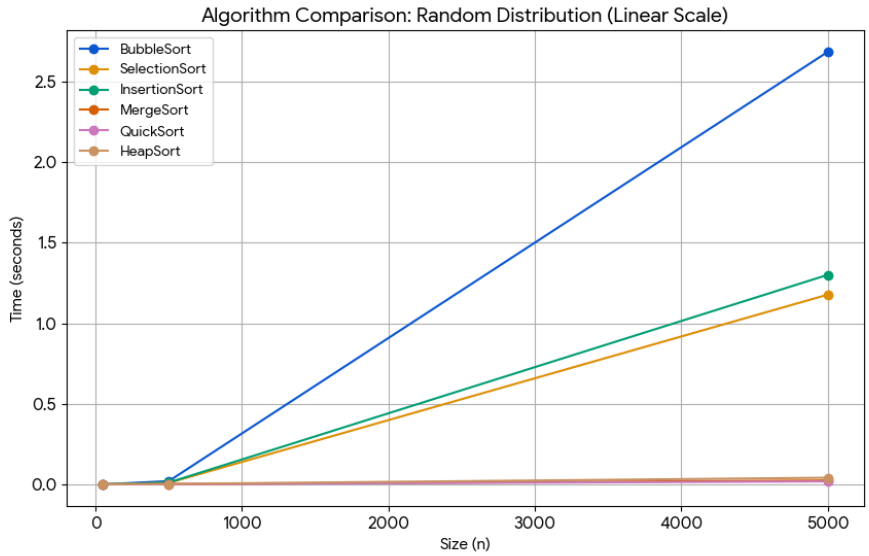

Figure 1: Linear comparison of all algorithms. The "wall" created by Bubble Sort at $n = 5,000$ demonstrates quadratic growth.

3.2 Efficient Algorithm Scaling ($n \leq 50,000$)

To better visualize the faster algorithms, Figure 2 uses a log-log scale. This allows us to compare Merge, Quick, and Heap Sort without them being "squashed" by the much slower $O(n^2)$ algorithms.

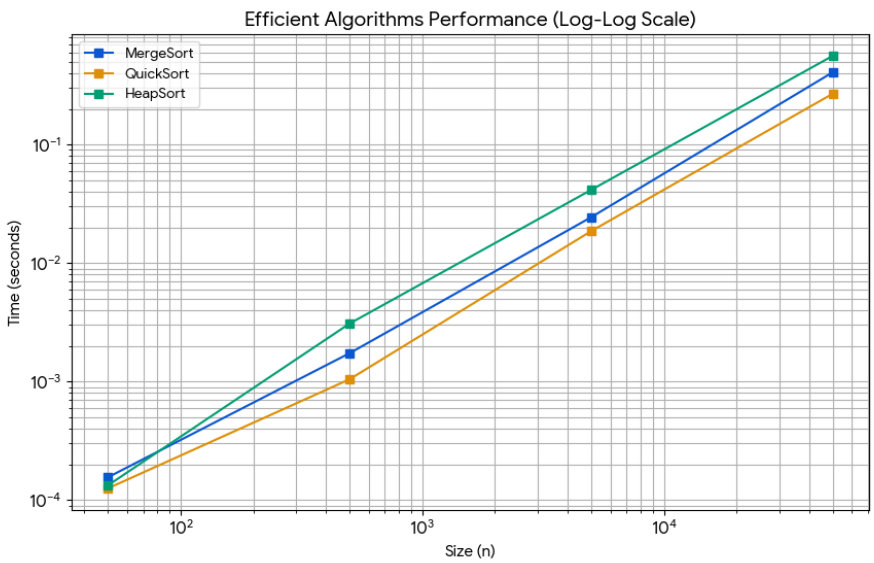

Figure 2: Log-Log scale comparison of efficient algorithms. The near-linear slope confirms stable $O(n \log n)$ behavior.

3.3 The QuickSort Distribution Anomaly

The most significant finding occurred at $n = 50,000$ with the **Flat** distribution. While QuickSort is generally the fastest, it is highly sensitive to the initial data distribution.

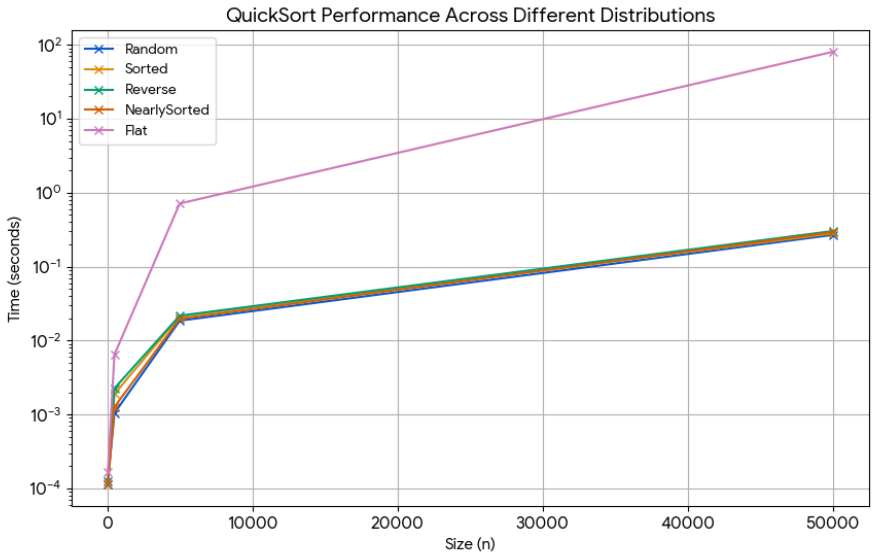

Figure 3: QuickSort performance across different distributions. Note the spike to 80.72s for Flat data.

While QuickSort averaged ≈ 0.27s on random data, it required **80.72 seconds** for the Flat distribution. This represents a degradation to $O(n^2)$ complexity, caused by inefficient pivot selection when handling high frequencies of duplicate values.

4 Conclusion

The experiment confirms that while QuickSort is often the fastest in practice for random data, it is vulnerable to specific distributions. Merge Sort and Heap Sort demonstrated the most consistent performance across all test cases.

5 Repository Link

The source code, raw datasets, and charts are available at:
`https://github.com/omar-lk0/Sorting_Algorithm_Comparison`